Assignment Code: DA-AG-012

# Decision Tree I Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is a Decision Tree, and how does it work in the context of classification?

**Answer:**

A **Decision Tree** is a non-parametric supervised learning algorithm used for both classification and regression tasks. It is structured like a flow-chart or an inverted tree, where the goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

**The structure of a decision tree**

- **Root Node:** The very first node at the top which represents the entire dataset. It is the starting point for splitting the data based on the most significant feature.
- **Decision Nodes:** These nodes represent a sub-group of the data and signify a decision point based on a specific feature or attribute.
- **Leaf Nodes**: These are the final nodes at the end of a branch that do not split further. In classification, a leaf node represents the final predicted class or category.
- **Branches:** The lines connecting nodes, representing the outcome of a decision or a path to the next node.

**How it Works in Classification**

1. **Selecting the Best Split**
    - Entropy
      H(s) = - $\sum_{k=0}^{n} pi \log pi$
    - Information Gain
    - Gini Impurity
2. **Splitting and Branching**
3. **Stopping Criteria & Pruning**

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

**Answer:**

**Gini Impurity**

Gini Impurity measures the frequency at which a randomly chosen element from the set would be incorrectly labeled if it was randomly labeled according to the distribution of labels in the subset.

**The Formula:**

**Gini = 1 -** $\sum_{k=0}^{n} (pi)^2$

**Impurity Levels:**

- For a **pure** node, Gini Impurity is **0**.
- For an **equally split** node, the maximum Gini Impurity is **0.5**.

**Impact on Splits:** The algorithm looks for the split that results in the **lowest Gini Impurity** for the child nodes. This is the default metric used by the **CART** (Classification and Regression Trees) algorithm in libraries like Scikit-Learn.

**Entropy**

Entropy measures the level of disorder, randomness, or uncertainty in a dataset.

- **The Formula:** It is calculated based on the probability (pi) of each class

  H(s) = - $\sum_{k=0}^{n} pi \log pi$

**Impurity Levels:** If a node is **pure** (all samples are one class), Entropy is **0**.

- If a node is **equally split** (50/50 mix), Entropy is at its maximum of **1.**

**Impact on Splits:** The algorithm calculates the Entropy of the parent node and the weighted average Entropy of the potential child nodes. The difference is called **Information Gain**. The tree chooses the split that results in the **highest Information Gain** (the largest reduction in uncertainty).

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

**Answer:**

In Pre-Pruning, you set "hyperparameters" before training starts to limit the tree's growth.

- **Practical Advantage:** Efficiency and Speed. Since the tree stops growing early, it saves significant computational power and memory. This is a massive advantage when you are working with very large datasets, like the Credit Card Fraud dataset (6M+ records) we discussed earlier.
- Common Parameters: max_depth: Limits how many levels the tree can have.
- min_samples_split: Minimum number of samples required to split a node.

**Post-Pruning (Reduced Error Pruning)**

In Post Pruning, you allow the tree to grow to its maximum possible depth (where every leaf is pure) and then "prune" the branches that don't add significant value.

- **Practical Advantage: Better Accuracy (Prevents Missing Patterns).** Some features might not look important at the top of the tree but become very significant when combined with other features deeper down. Pre-pruning might "kill" these branches too early, but Post pruning keeps them if they actually help in the final classification

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

**Answer:**

Information Gain is the mathematical metric used to determine which feature or attribute provides the most "clarity" when splitting data. It is essentially a measure of how much a particular split reduces the Entropy (uncertainty) of a dataset.

**The Information Gain is the result of subtracting the weighted average of the children's entropy from the parent's entropy.**

***Information Gain = Entropy(Parent) - [Weighted Average] * Entropy(Children)***

**Why It Is Important for Choosing the Best Split**

Information Gain acts as the primary "decision-maker" for the tree's architecture for several critical reasons:

- **Purity Selection:** The algorithm evaluates every possible feature and chooses the one with the **highest Information Gain**. A higher gain means the resulting groups are "purer" and contain more similar data points.
- **Tree Efficiency:** By choosing the split that provides the most information at each step, the algorithm creates a more compact and efficient tree. This prevents the tree from growing unnecessarily large or complex.

- **Predictive Power:** Features that have the highest Information Gain are often the most important predictors in your dataset. For your **Credit Card Fraud** project, Information Gain would help identify if Transaction Amount or Location is a stronger indicator of fraud.

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

**Answer:**
**Real-World Applications of Decision Tree**
1. Financial Services (Fraud & Risk)
2. Healthcare (Diagnosis)
3. E-commerce (Customer Churn)
4. Manufacturing (Quality Control)

**Main Advantages**
- Easy to Understand & Interpret
- No Data Scaling Required
- Handles Mixed Data
- Automatic Feature Selection

**Main Limitations**
- Overfitting
- Instability
- Biased toward High-Level Categories
- Greedy Approach

Dataset Info:

- Iris Dataset for classification tasks (sklearn . datasets . load_iris( ) or provided CSV).

- Boston Housing Dataset for regression tasks (sklearn . datasets . `load_boston(` ) or provided CSV).

Question 6: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier using the Gini criterion
- Print the model's accuracy and feature importances

(Include your Python code and output in the code box below.)

**Answer:**

```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load the Iris Dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Train a Decision Tree Classifier using the Gini criterion
# 'gini' is the default criterion for DecisionTreeClassifier in sklearn
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)

# 3. Print the model's accuracy
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy * 100:.2f}%")

# 4. Print Feature Importances
print("\nFeature Importances:")
for name, importance in zip(feature_names, clf.feature_importances_):
    print(f"{name}: {importance:.4f}")
```

```
Model Accuracy: 100.00%

Feature Importances:
sepal length (cm): 0.0000
sepal width (cm): 0.0167
petal length (cm): 0.9061
petal width (cm): 0.0772
```

Question 7: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier with max_depth=3 and compare its accuracy to a fully-grown tree.

(Include your Python code and output in the code box below.)

**Answer:**

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load the Iris Dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 2. Train a Fully-Grown Decision Tree (No depth limit)
full_tree = DecisionTreeClassifier(random_state=42)
full_tree.fit(X_train, y_train)
full_acc = accuracy_score(y_test, full_tree.predict(X_test))

# 3. Train a Pruned Decision Tree (max_depth=3)
pruned_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
pruned_tree.fit(X_train, y_train)
pruned_acc = accuracy_score(y_test, pruned_tree.predict(X_test))

# Print Comparison
print(f"Accuracy of Fully-Grown Tree: {full_acc * 100:.2f}%")
print(f"Accuracy of Pruned Tree (max_depth=3): {pruned_acc * 100:.2f}%")
print(f"Nodes in Fully-Grown Tree: {full_tree.tree_.node_count}")
print(f"Nodes in Pruned Tree: {pruned_tree.tree_.node_count}")
```

```
Accuracy of Fully-Grown Tree: 100.00%
Accuracy of Pruned Tree (max_depth=3): 100.00%
Nodes in Fully-Grown Tree: 19
Nodes in Pruned Tree: 9
```

Question 8: Write a Python program to:

- Load the Boston Housing Dataset
- Train a Decision Tree Regressor
- Print the Mean Squared Error (MSE) and feature importances

(Include your Python code and output in the code box below.)

**Answer:**

```python
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
# 1. Load the Boston Housing Dataset
# Using fetch_openml to retrieve the dataset by its name
boston = fetch_openml(name='boston', version=1, as_frame=True, parser='auto')
X = boston.data
y = boston.target
# Split the data into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Train a Decision Tree Regressor
# We use the Regressor instead of the Classifier for continuous value prediction
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X_train, y_train)
# 3. Predict and calculate Mean Squared Error (MSE)
y_pred = regressor.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
# 4. Print the MSE and Feature Importances
print(f"Mean Squared Error (MSE): {mse:.2f}")
print("\nFeature Importances:")
features = boston.feature_names
importances = regressor.feature_importances_
# Display features sorted by importance
feature_importance_df = pd.DataFrame({'Feature': features, 'Importance': importances})
print(feature_importance_df.sort_values(by='Importance', ascending=False))
```

```
Mean Squared Error (MSE): 10.42

Feature Importances:
    Feature  Importance
5        RM    0.600326
12    LSTAT    0.193328
7       DIS    0.070688
0      CRIM    0.051296
4       NOX    0.027148
6       AGE    0.013617
9       TAX    0.012464
10  PTRATIO    0.011012
11        B    0.009009
2     INDUS    0.005816
1        ZN    0.003353
8       RAD    0.001941
3      CHAS    0.000002
```

Question 9: Write a Python program to:

- Load the Iris Dataset
- Tune the Decision Tree's max _ depth and min _ samples _ split using GridSearchCV
- Print the best parameters and the resulting model accuracy

(Include your Python code and output in the code box below.)

**Answer:**

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# 1. Load the Iris Dataset
iris = load_iris()
X, y = iris.data, iris.target
# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# 2. Define the model and the parameter grid
dtree = DecisionTreeClassifier(random_state=42)

param_grid = {
    'max_depth': [2, 3, 4, 5, 6, None],
    'min_samples_split': [2, 5, 10, 20]
}
# 3. Tune using GridSearchCV
# cv=5 means 5-fold cross-validation
grid_search = GridSearchCV(estimator=dtree, param_grid=param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
# 4. Print the best parameters and resulting model accuracy
print(f"Best Parameters: {grid_search.best_params_}")
# Evaluate the best model on the test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
test_accuracy = accuracy_score(y_test, y_pred)
print(f"Best Training Score (Mean CV): {grid_search.best_score_ * 100:.2f}%")
print(f"Final Test Accuracy: {test_accuracy * 100:.2f}%")
```

```
Best Parameters: {'max_depth': 4, 'min_samples_split': 2}
Best Training Score (Mean CV): 94.17%
Final Test Accuracy: 100.00%
```

Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease. You have a large dataset with mixed data types and some missing values.

Explain the step-by-step process you would follow to:

- Handle the missing values • Encode the categorical features
- Train a Decision Tree model
- Tune its hyperparameters
- Evaluate its performance

  And describe what business value this model could provide in the real-world setting.

**Answer:**

Step 1: Handle Missing Values

In a healthcare setting, missing data can be "Missing at Random" (e.g., a test wasn't conducted) or "Informative" (e.g., the patient was too ill for the test).

- Numerical Data: Use Median Imputation if the data has outliers (common in blood pressure or sugar levels) or Mean Imputation if it follows a normal distribution. For more accuracy, use KNN Imputer to estimate missing values based on similar patients.

- Categorical Data: Use Mode Imputation (most frequent value) or create a new category called "Missing" to see if the absence of data itself is a predictor of the disease.

Step 2: Encode Categorical Features

Machine learning models require numerical input.

- Ordinal Encoding: Use this for features with a natural order (e.g., Disease Stage: Stage 1, Stage 2, Stage 3).
- One-Hot Encoding: Use this for nominal features without order (e.g., Gender, Blood Type, or Symptoms like "Cough" vs "Fever"). This creates binary (0 or 1) columns for each category.

3. Train a Decision Tree Model

- Data Splitting: Divide the dataset into Training (80%) and Testing (20%) sets.
- Model Selection: Initialize the DecisionTreeClassifier using the Gini Impurity or Entropy criterion to measure split quality.
- Fitting: Train the model on the cleaned, encoded training data so it learns the decision rules (e.g., "If Hemoglobin < 12 and Age > 50, then Disease = Yes").

4. Tune Hyperparameters

To avoid overfitting and ensure the model works on new patients, use GridSearchCV:

- max_depth: Limit the depth to prevent the tree from becoming too complex.
- min_samples_split: Set a minimum number of samples required to split a node.
- ccp_alpha: Use this for Post-Pruning to simplify the tree after it is fully grown.

5. Evaluate Performance

In healthcare, a simple "Accuracy" score is not enough. You must look at:

- Recall (Sensitivity): This is the most critical metric. It tells you what percentage of actual sick patients were correctly identified. Missing a sick patient (False Negative) is much more dangerous than a false alarm.
- Precision: What percentage of predicted disease cases were actually correct.
- Confusion Matrix: To visualize the True Positives, True Negatives, False Positives, and False Negatives.

**Real-World Business Value**

1. **Early Detection & Intervention:** The model can flag high-risk patients before severe symptoms appear, allowing for earlier treatment which is cheaper and more effective.
2. **Resource Optimization:** Hospitals can prioritize lab tests and specialist appointments for patients the model identifies as "High Risk," reducing wait times and operational costs.
3. **Standardized Diagnosis:** The model provides a consistent, data-driven "second opinion" for doctors, reducing the risk of human error or bias in diagnosis.
4. **Reduced Hospital Readmissions:** By predicting which patients are likely to fall ill again, healthcare providers can plan better follow-up care, saving costs associated with emergency readmissions